

netimpute: Joint Multiple Imputation of Node Attributes and Network Ties in R

Robert W. Krause

Abstract

Missing data in social network studies routinely affects both node-level attributes and the network ties themselves. Standard multiple imputation software such as **mice** (van Buuren and Groothuis-Oudshoorn, 2011) handles the former well but has no notion of network structure, while dedicated network-imputation procedures typically treat tie imputation and attribute imputation as separate problems. This paper introduces **netimpute**, an R package that (a) computes a broad battery of node-level structural and attribute homophily measures across one or more networks, (b) can reduce these to a manageable set of principal-component predictors while optionally preserving specific raw measures that are themselves part of substantive hypotheses, (c) provides a dyadic (cell-level) regression for network ties in the spirit of MR-QAP (Krackhardt, 1988), and (d) combines both into **netmice()**, a chained-equations routine that jointly imputes missing node attributes and missing network ties by cycling between them, using network-derived predictors for attributes and attribute-derived (and other-network-derived) predictors for ties. Network ties are updated *tie-wise* by default: a sequential Gibbs step redraws each missing tie one at a time, conditional on all previously imputed ties, refreshing the endogenous statistics (reciprocity, shared contacts) after every single draw via change statistics. The package additionally supports structural zeros, logical constraints between networks, social-relations-model random intercepts in the tie model, custom imputation models with interactions among any internally created terms, and network-aware per-target predictor selection (**netquickpred()**). We describe the package’s design choices, the measures it computes for binary and non-negative weighted networks, and illustrate its use with a reproducible example.

Keywords: multiple imputation, missing data, social network analysis, R, predictive mean matching, dyadic regression, MR-QAP, Gibbs sampling.

1 Introduction

Two kinds of missingness arise in studies involving networks, which are usually handled by different tools. Missing values on *node attributes* (age, performance, survey responses, etc.) are the domain of general-purpose multiple imputation (MI) software, most prominently **mice** (van Buuren and Groothuis-Oudshoorn, 2011), which implements the fully conditional specification / chained equations approach to MI (Rubin, 1987; Little and Rubin, 2002) or model based likelihood treatments (e.g., FIML). Missing *network ties* are instead the subject of a smaller, more specialized literature on network imputation (Huisman, 2009; Koskinen et al., 2010; Krause et al., 2018, 2020), often developed alongside exponential random graph models (Robins et al., 2007) or Stochastic Actor-oriented Models (Snijders, 1996, 2017) rather than as an extension of standard MI. A notable exceptions to this separation are Koskinen et al. (2013) and Krause (2019), which both propose joint imputation of attributes and ties, but neither is implemented as a general-purpose, formula-extensible R package. They are also limited in the number of network relations and attributes they can handle.

In practice, these two kinds of missingness are rarely independent problems. A respondent who did not answer the attribute survey is often the same respondent whose outgoing nominations are also missing; and the node-level measures researchers use to *explain* attributes (e.g., is a

central employee also a high performer?) are themselves undefined until the network is complete, while the covariates used to predict a missing tie (e.g., do two students share a gender or interest?) are undefined until the attributes are complete. Imputing one without regard to the other risks two failure modes: treating an incomplete network as if it were complete when computing network derived predictors for attribute imputation, and treating incomplete attributes as if observed when building the dyadic covariates used to impute ties. Any imputation focused only on one side of this problem works only conditional on the other missing data being irrelevant.

netimpute addresses this by combining the imputation of attributes and network ties together in one function. While the imputation of the networks is not as thorough as a full model-based approach (e.g., using SAOMs or ERGMs), it is designed to be fast and scalable to multiple networks and attributes, while still capturing the essential dependencies between them.

The remainder of this paper is organised as follows. Section 2 briefly situates **netimpute** relative to existing R tools for network analysis and for missing data. Section 3 describes the package’s building blocks: node-level measures with a selectable measure set (`net_measures()` and its `net_measures_core()` / `net_measures_full()` wrappers), predictor construction for imputation (`net_predictors()`), dyadic regression (`dyad_regression()`), per-target predictor selection (`netquickpred()`), and the joint imputation routine (`netmice()`). Section 4 works through a reproducible example. Section 5 discusses current limitations and directions for future work.

2 Related work

R’s network analysis ecosystem is mature: **igraph** (Csardi and Nepusz, 2006) and **sna** (Butts, 2008) both provide extensive node- and network-level measures, and **netimpute** builds directly on **igraph** for its graph representation and uses **sna** as an optional source of a handful of additional centrality measures (Gil-Schmidt power, flow/load/stress centrality, information centrality, and prestige) that **igraph** does not provide. Neither package, however, is designed for missing data: both assume a fully observed network. The same is true for most functions in the new **manynet** package (Hollway, 2021) and its sister packages.

On the imputation side, **mice** (van Buuren and Groothuis-Oudshoorn, 2011) is the standard R implementation of chained-equations MI and is used internally by **netimpute** as the engine for the actual univariate imputation step. By default it uses predictive mean matching for numeric and binary targets, with **mice**’s other numeric-response univariate methods selectable through the same `method` argument and multinomial logistic regression via `mice::mice.impute.polyreg()`, which in turn relies on **nnet** (Venables and Ripley, 2002), for nominal attributes with more than two categories.

Prior work on missing network data (Huisman, 2009; Krause et al., 2020) has established that simple treatments (listwise deletion, treating a missing tie as absent) can materially bias network-level statistics, and has proposed imputation procedures based on the observed structure. These procedures are not implemented as a general-purpose, formula-extensible R packages; **netimpute**’s dyadic regression step can be seen as a simple, point-prediction analogue of MR-QAP (Krackhardt, 1988) used specifically to build a predictive (rather than inferential, and thus avoiding the need for permutation) model for tie imputation. The default tie-wise updating scheme inside `netmice()` (Section 3.5) additionally borrows from the exponential-random-graph-model tradition (Robins et al., 2007): each missing tie is redrawn from its full conditional distribution given the current state of every other tie — a Gibbs update of an ERGM-style model whose parameters are estimated by pseudo-likelihood — while remaining restricted to the missing cells, so observed ties are never resampled. This is not seen as a replacement or improvement upon proper ERGM-based (or SAOM-based) imputations, but rather as a fast, scalable, and easy to use way to capture the essential dependencies between ties, attributes, and networks for the purpose of imputation.

3 The netimpute package

3.1 Node-level measures

A network carries a lot of information that can be useful for imputing not just itself but also missing attribute values. The function `net_measures_core(net, attributes, attr_types = NULL, id_col = NULL)` computes nine structural measures per node — out-degree, in-degree, a reciprocity measure, Bonacich power centrality (with an automatically rescaled exponent so it converges regardless of the network’s spectral radius), normalised betweenness, an isolate indicator, Burt’s constraint (Burt, 1992), harmonic closeness, and local clustering — together with, for every `attribute`, a scale-appropriate homophily/alter-similarity block: an E-I index, Blau’s index, and the modal alter category for binary or multinomial attributes; average absolute difference on outgoing and on incoming ties, the mean, minimum, and maximum alter value, plus the mean alter value over incoming and over outgoing ties separately, for continuous attributes. `net_measures_full()` extends this to 28 structural measures (the full degree family, eigenvector/alpha/PageRank centrality, structural holes measures, coreness, and, if `sna` is installed, six additional measures) using exactly the same homophily block.

Both functions are wrappers around `net_measures(net, attributes, measure_set)`: `measure_set` accepts any mix of the named sets "core" and "full", the per-attribute "homophily" block, and individual measure names, and returns their union — e.g. `measure_set = c("core", "total_degree", "pagerank")` computes the core battery plus two extras, while `c("indegree", "betweenness")` computes exactly those two measures and nothing else. Every implemented measure is enumerated in the `net_measures()` and `netmice()` help pages. The same `measure_set` argument is accepted by `net_predictors()`, `netquickpred()`, and `netmice()`, which use these measures as attribute-imputation predictors.

Only binary (0/1) and non-negative weighted networks are currently supported. Supplying a network with a negative tie value raises an error and it is currently recommended to split signed networks into two non-negative networks (e.g., “friendship” and “antagonism”) for separate imputation, and recombine them afterwards (Section 5). For non-negative weighted networks, two measures are redefined: reciprocity of a node becomes the average $|w_{ij} - w_{ji}|$ over alters tied in either direction, and local clustering becomes the geometric-mean weighted clustering coefficient of Opsahl and Panzarasa (2009): for node i and each pair of its neighbours (j, k) , the two-path value $\sqrt{w_{ij}w_{ik}}$ is summed over every such pair to form a denominator, and over only the pairs where the triangle is actually closed (j and k are themselves tied) to form the numerator. This ratio reduces exactly to the binary clustering coefficient when all weights equal 1.

3.2 Predictors for imputation

`net_predictors()` applies `net_measures_core()` or `net_measures_full()` across a list of networks (and a matching list of attribute data frames), stacks the results, and offers three output modes: `output = "measures"` returns the stacked raw measures. `output = "pca"` instead returns as many leading principal components of the measure matrix as the PCA budget allows, addressing the multicollinearity that is otherwise inevitable once a dozen or more structural and homophily measures are computed per network and the inflation of predictors, which can quickly exceed the N in small networks. `output = "both"` returns the principal components *and* a user-specified set of raw measures (`keep_vars`), preserved untransformed; this is for the situation where a specific measure — say, out-degree — is itself part of a hypothesised relationship (i.e., a predictor in a regression) the analyst does not want diluted by including it into a composite component, which can also be residualised on the remaining predictors so the components capture variance orthogonal to it (`residualize_kept = TRUE`).

3.2.1 The predictor budget

How many predictors any single imputation model may carry is governed by one argument, `PCA`, a list with `n` (a fixed ceiling) and/or `ratio` (observations required per predictor); when both are given the smaller budget binds. Whatever the budget, terms that must keep their own named coefficient — a network’s own `reciprocity` and `twopath`, the cross-network cell terms, the `isolate` indicator, and anything named in `models` — are exempt, and the budget applies to the remainder.

The denominator of `ratio` is deliberately not the row count. For a binary target, which includes every tie model, it is the *minority class count* among the observed cells. A logistic model is limited by its rarer outcome rather than by its sample size: a 30-node network at density 0.15 presents roughly 870 dyadic rows but only about 130 ties, so budgeting against the rows would licence enough predictors to separate the data perfectly. The default `ratio = 10` is the conventional events-per-variable floor (Peduzzi et al., 1996; Harrell, 2015). Continuous and multinomial targets use the observed row count.

3.3 Dyadic regression

`dyad_regression()` vectorises a target network’s adjacency matrix (excluding the diagonal) and regresses it for every node attribute on an ego value, an alter value, and either an absolute difference (continuous attributes) or a same-category indicator (categorical attributes), reciprocity (the transposed target network), a bounded 0/1 two-path indicator (`twopath` — “at least one shared contact”, i.e. at least one path $i \rightarrow k \rightarrow j$ to impute x_{ij}), and, for every *other* network supplied, its tie value, its reciprocity (transpose), and the sender’s and the receiver’s out- and in-degree in that network. When many other networks are supplied, `other_net_predictors = "pca"` replaces the node-level degree terms with the first `n_components` principal components. By default the dyad-level cross-network terms (the other network’s cell and its transpose) are kept as predictors and not taken into the PCA to preserve the direct relationship between the networks. The complete list of term names, as referenced from `netmice()`’s `models` formulas, is documented in the `dyad_regression()` help page. Dyads marked *structurally absent* (zero by design, e.g., nominations the study design made impossible; argument `structural`) are removed internally from the dyad data entirely, like the diagonal. The model is fit by `glm()` with a user-specified `family` (default Gaussian). Alternatively, `random_intercepts` fits the model with `lme4`, adding social-relations-model-style random intercepts per sender (`"ego"`), per receiver (`"alter"`), and/or per unordered pair (`"dyad"`; directed networks only — for an undirected target the two rows of a dyad are duplicates and the intercept degenerates, which triggers a warning, and the fixed reciprocity term is dropped when `"dyad"` is used since the dyad effect *is* the SRM reciprocity term and is exactly singular with it). Unlike in MR-QAP (Krackhardt, 1988), permutation is not necessary for this regression, since the goal here is prediction for imputation rather than a permutation-based significance test/inference.

3.4 Per-target predictor selection

`netquickpred()` is a network-aware, recursive generalisation of `mice::quickpred()`. For every target with missing values it screens candidate predictors by absolute pairwise-complete correlation — with the target’s observed values and, like `mice::quickpred()`, with its missingness indicator — against a threshold `mincor` (default 0.1). For attribute targets the candidates are the other attributes plus every node-level network measure of Section 3.1 (computed on the observed ties); for network targets, screening happens at the dyad level on the design terms of Section 3.3, with the target’s own `reciprocity` and `twopath` always included unscreened.

Because a selected predictor’s own missing values are imputed inside the chain, the selection then recurses: with `steps = 3` (default), the direct predictors, their predictors, and the predic-

tors of those all enter the target’s model. Finally, near-duplicate survivors are pruned per target, either by pairwise correlation (default, threshold 0.9) or by variance-inflation factor (threshold 10), with a network target’s endogenous predictors exempt from pruning. The result object reports, for every target, exactly which predictors were selected at which step and with what screening correlation; handed to `netmice()` (or requested there directly via `predictor_selection = "quickpred"`), it replaces the everything-plus-PCA default of Section 3.5 with target-specific models in which every coefficient keeps its own name. A companion argument, `targets`, names the variables/networks whose imputations are actually needed: everything else with missing data that is not selected as a (recursive) predictor, target, or explicitly included in an imputation model is dropped from the imputation algorithm entirely, saving computation time and avoiding that the model gets flooded with irrelevant predictors.

3.5 Joint imputation

`netmice()` is a chained-equations routine, architecturally modelled on `mice`’s own sampler, generalised to cycle through both attributes and networks. Within one iteration, every attribute and every network with missing values is visited exactly once, in a single interleaved sequence whose order is randomised independently for each of the m imputation chains. Visiting an attribute rebuilds its network-derived predictors (Section 3.1) from whichever networks were most recently updated; visiting a network rebuilds its dyadic predictors (Section 3.3) from whichever attributes and other networks were most recently updated. Because these predictors depend on *the other* data type, they are never cached: a variable that depends on both a network and an attribute is effectively recomputed at both points in the cycle.

Each chain starts from a conservative state: missing attribute values are initialised by re-sampling observed values, but missing ties are initialised to 0 (“no tie”; `net_init = "zero"`), so the first sweep’s endogenous predictors — reciprocity, two-paths, other networks’ tie and degree terms — are built from observed ties only. This biases the very first sweep’s structural terms downward, but avoids seeding a self-reinforcing surplus of random initial ties that later sweeps amplify. A density-based initial imputation is available as `net_init = "sample"`.

Tie-wise updating. A network visit updates its missing cells *tie-wise* by default (`net_update = "gibbs"`). A working model is fit once per visit — logistic regression for a binary network, linear regression otherwise — and one coefficient vector is drawn from its asymptotic posterior, mirroring `mice`’s Bayesian β -draw so that between-imputation variability reflects estimation uncertainty. The missing cells are then visited one at a time in random order; each cell’s linear predictor is evaluated with the *current* values of the endogenous statistics. A binary tie is then drawn from $\text{Bernoulli}(\text{logit}^{-1}(\eta))$; a weighted tie by predictive mean matching against the observed dyads’ fitted values. Each draw therefore conditions on all previously imputed ties: this is exactly the full-conditional (Gibbs) update of an ERGM whose parameters were estimated by pseudo-likelihood restricted to the missing cells. It yields properly dependent within-network imputations and avoids potential synchronous-update artifacts of the alternative scheme, `net_update = "simultaneous"`, in which one model per visit imputes every missing cell at once from the same snapshot of the statistics (e.g., if both x_{ij} and x_{ji} are missing, their imputation would be independent of each other).

Undirected networks. In an undirected network the cells x_{ij} and x_{ji} are not two ties but two records of the same tie, and imputing them independently would return a matrix that is no longer symmetric. This is more than an aesthetic defect: throughout the package directedness is detected from the matrix itself, so a single desymmetrized pair reclassifies the network as directed for every node- and dyad-level measure derived from it in the remaining sweeps, and in the returned object. A network is therefore treated as undirected whenever the supplied matrix

is symmetric in its values *and* in its pattern of missing cells — precisely the case in which the two cells record one tie — and is then imputed one tie at a time.

This classification is established once, from the incomplete data supplied to `netmice()`, and is thereafter used wherever directedness matters: the tie updater, the initialisation, the node-level measures that serve as attribute predictors, and the per-iteration diagnostics. It deliberately is not re-derived from the working state, because filling destroys the evidence it rests on. A directed network whose asymmetry lies entirely in its missing cells becomes symmetric as soon as those cells are initialised at zero; a measure recomputed from that state would classify it as undirected and, for instance, return the per-node reciprocity ratio as missing for every node, quietly dropping a genuine predictor from that visit.

Under tie-wise updating each unordered pair is visited once. Both directions' linear predictors are evaluated against the current state and averaged, on the probability scale for a binary tie, $p = \frac{1}{2} (\text{logit}^{-1}(\eta_{ij}) + \text{logit}^{-1}(\eta_{ji}))$, and on the linear predictor scale for a weighted one; the single resulting draw is written to both cells. The two predictors generally differ even for an undirected target, because the dyad-level design retains the sender/receiver-specific terms of the other (possibly directed) networks and of the nodal attributes, so averaging them, rather than arbitrarily privileging one direction, keeps the update symmetric in the two nodes. Under the simultaneous scheme the mirror cells are necessarily drawn from the same snapshot and can disagree; each disagreeing pair is then reconciled after the draw, by keeping one of the two values at random for a binary network — each is an equally valid draw of the same tie — and by averaging them for a weighted one. Observed cells are never overwritten by either step.

Under the simultaneous scheme, the tie working model can instead be a linear mixed model (`net_random_intercepts`): random intercepts per sender, receiver, and/or unordered dyad, fit with `lme4`, capture actor-level activity/popularity heterogeneity beyond the degree-based fixed effects, as in the social relations model, with missing ties imputed by predictive mean matching on the conditional fitted values. Random intercepts require the simultaneous scheme: setting `net_random_intercepts` while leaving `net_update` at its default falls back to "`simultaneous`" with a message, whereas combining it with an explicit `net_update = "gibbs"` is an error. Note of course that using random intercepts is only meaningful if missing ties are tie non-response and not actor non-response (Huisman and Steglich, 2008), since the latter means that all data for the random intercept of a missing node are missing.

Structural zeros and between-network constraints. Two further arguments restrict where ties may exist at all. `structural` marks cells that are zero *by design* (e.g., nominations into another school class that respondents could not make): such cells are excluded from the initialisation donor pool, and removed from the dyad-level regression rows, so design-zeros cannot inflate the zeros of the model. `net_dependence` declares *adaptive* logical constraints between networks: `necessary` rules ("a tie in *A* is required for a tie in *B*") and `forbidden` rules ("ties in *A* and *B* are mutually exclusive"). Before the chains, missing cells already determined by observed cells are deduced and filled (with observed contradictions raising an error); at every visit of a constrained network, its cells currently determined to be 0 by the partners' *current* — possibly imputed — state are treated exactly like structural cells for that visit (i.e., fixed to zero and removed from the estimation); and after every network re-imputation the rules are followed, so every completed network list satisfies every rule.

Attributes. An attribute's predictor set includes the homophily block built from the target attribute itself, not only from the other attributes: under homophily or social influence, the composition of a node's alters on the variable being imputed (its mean alter value, and in particular the mean over *incoming* ties, which rest on other — typically observed — nodes' reports) is among the strongest predictors available. The only measures withheld when imputing an attribute are those that are direct functions of ego's own current value of that attribute (its

E-I index and the average-absolute-difference terms), since regressing a variable on a transform of itself would anchor imputed values to their own previous draws rather than to information from the rest of the data.

The actual univariate draw for an *attribute* is delegated to **mice**, generically: **method** names any of **mice**'s numeric-response univariate methods and is dispatched to the corresponding **mice.impute.<method>()** function — predictive mean matching ("**pmm**", the default), its distance-weighted variant "**midastouch**", random sampling from the observed values ("**sample**"), classification and regression trees ("**cart**"), random forests ("**rf**"; requires **ranger** or **randomForest**), the Bayesian and bootstrap linear-regression family ("**norm**", "**norm.nob**", "**norm.boot**", "**norm.predict**"), and "**mean**" (the latter two are deterministic and therefore improper for multiple imputation, as **mice**'s own documentation cautions). **method** may also be a named character vector assigning *per-target* methods, **mice**-style: named entries set the method for individual attributes or networks, and at most one unnamed entry sets the default for the rest — e.g. **method = c("cart", performance = "norm")** imputes **performance** by Bayesian linear regression and everything else by trees. Entries naming a multinomial attribute, or a network whose ties are updated tie-wise or with random intercepts, are ignored with a message, since those targets' working models are fixed by design.

Because the auto-generated predictor set can plausibly approach or exceed the number of observed cases available for a given target — several networks' worth of structural and homophily measures adds up quickly relative to a modest number of nodes — **netmice()** follows the same variable reductions as described above.

A specific, hypothesised relationship can be protected from dilution or omission via the **models** argument, a list of formula strings whose left-hand side names the attribute or network they apply to; the right-hand side is evaluated with **model.matrix()** (so interaction syntax such as **performance * gender** works) and its columns are *appended to*, never substituted for, the auto-generated predictors. A network's own formula must reference the dyadic predictor names from Section 3.3 (e.g., **age_absdiff**, **advice_tie**), not raw node attributes, and may interact any internally created term with any other — including the target's own endogenous terms and other networks' terms: **friends ~ age_ego:reciprocity**, for instance, lets the tendency to reciprocate depend on the sender's age. Under the default tie-wise updating, an interaction involving **reciprocity** or **twopath** is, like those main effects, re-evaluated from the current matrix state after every single tie draw, so the interaction stays live throughout the sequential sweep. The full lists of referenceable dyad-level term names and of implemented network measures are given in dedicated sections of the **netmice()** help page. Variables and networks a formula depends on are automatically protected from being dropped by the **targets** mechanism, so the formula stays evaluable at every visit.

For **ncores > 1**, the *m* chains run as independent **future** futures (**future::multisession**), each with its own random visit order and an explicitly pinned random-number generator so that results are identical whether run sequentially or in parallel given the same **seed** (multisession workers otherwise default to a different RNG algorithm for proper parallel streams, which would silently change results if the seed value were reused as-is). **netmice()** returns an object of class **netmids** recording, in parallel to **mice**'s own convergence diagnostics, the mean and variance of each imputed attribute value across iterations, and, for every network, its density, reciprocity, transitivity, isolate count, and average inverse geodesic distance at every iteration. Because those network statistics are computed on the *completed* network, observed ties included, they are diluted by the missingness fraction and can look flat while the imputed portion drifts; the mean and variance of the imputed ties alone are therefore tracked separately and are the more sensitive diagnostic.

In addition, **netmice()** reports rank-normalized split- $\hat{R}$ (Gelman and Rubin, 1992; Vehtari et al., 2021) for each tracked quantity, computed after discarding the first half of the iterations, and warns when any value exceeds 1.05. Constant traces return **NA** rather than a spurious

statistic. Users should note that $\hat{R}$ is a large-sample diagnostic: at the default `maxit`, the retained draws per half-chain number in the single digits, and the statistic is correspondingly noisy in both directions.

3.6 Data entry: matrices or an edgelist

Adjacency matrices are rarely the form in which network data arrives, so `networks` also accepts a single data frame `edgelist`, described by `edgelist_options`. One edgelist may define many networks: `edgelist_format = "long"` splits it on the interaction of the columns named in `edgelist_split`, producing one network per observed combination (so waves crossed with name-generator questions yield a network per wave-question pair), while `"wide"` treats each named column as a separate network holding that tie's value or NA.

Because an edgelist carries node names rather than positions, the `id` argument names the column of `data` holding those identifiers; it is required for edgelist input and optional, though advisable, for matrices, where it matches and reorders their dimnames. Two further options carry the information an edgelist structurally cannot. `nodelist` supplies each network's roster, without which isolates and wholly non-responding actors would be invisible; a node off a network's roster is treated as structurally absent there rather than as having no ties. `missing` names the actors whose ties are unknown — by default their *outgoing* ties, the ordinary actor non-response case (Huisman and Steglich, 2008) — so that their unlisted cells become missing rather than observed zeros. Nodes appearing in the edgelist but not in `data` are added with all attributes missing, and reported.

The distinction matters because an edgelist records only which ties exist: every unlisted cell is a zero by omission, and nothing in the format distinguishes “no tie” from “not asked”. `netmice()` therefore warns when an actor is marked missing in a direction in which the edgelist nonetheless lists ties for them, and directs the user to the wide format or to matrices, which can express partial observation. The warning is direction-matched: an actor whose outgoing ties are missing is still expected to appear as a receiver, since others nominated them.

4 Illustrative example

The example below simulates a 40-node organisation with two networks (a binary friendship network and a Poisson-weighted advice network) and four attributes spanning all three supported scales (continuous, binary, multinomial). Numbers below are the output of running this code with R 4.6.1 and `netimpute` 0.1.0; the full script (which also shows the informational messages about inferred attribute types omitted here) ships as `paper/example_run.R`. Output lines longer than the page width are re-wrapped for display.

```
set.seed(20260701)
n <- 40
friends <- matrix(rbinom(n * n, 1, 0.08), n, n); diag(friends) <- 0
advice <- matrix(rpois(n * n, 0.35), n, n); diag(advice) <- 0

attrs <- data.frame(
  age = round(rnorm(n, 34, 7)),
  gender = sample(c("F", "M"), n, replace = TRUE),
  department = sample(c("sales", "eng", "hr"), n, replace = TRUE,
    prob = c(0.4, 0.4, 0.2)),
  performance = round(rnorm(n, 50, 10), 1)
)

core_out <- net_measures_core(friends, attrs)
head(core_out[c("node_id", "outdegree", "indegree",
  "reciprocity_ratio", "betweenness", "constraint")])
```


	node_id	outdegree	indegree	reciprocity_ratio	betweenness	constraint
1	1	5	4	0.125	0.095	0.154
2	2	3	1	0.000	0.047	0.302
3	3	2	1	0.000	0.005	0.333
4	4	3	3	0.000	0.062	0.187
5	5	1	0	0.000	0.000	1.000
6	6	2	3	0.000	0.048	0.200

Any other subset of the implemented measures can be requested through `net_measures()` directly (the ‘attributes’ argument is only needed when the homophily block is among the requested measures):

```
names(net_measures(friends, measure_set = c("indegree", "betweenness",
                                             "pagerank")))
```

```
[1] "node_id"      "indegree"      "betweenness"   "pagerank"
```

A dyadic regression of friendship ties on the node attributes plus the advice network illustrates the predictor set from Section 3.3 (selected coefficients shown; this network was simulated with no true structure, so none of these are significant, as expected):

```
dr <- dyad_regression(list(friends = friends, advice = advice),
                        attrs, target = "friends")
summary(dr$model)$coefficients[
  c("age_absdiff", "department_same", "advice_tie"), ]
```

	Estimate	Std. Error	t value	Pr(> t)
age_absdiff	0.0001873604	0.001025267	0.18274298	0.8550238
department_same	-0.0085523890	0.014095721	-0.60673652	0.5441154
advice_tie	-0.0009255629	0.011364711	-0.08144184	0.9351012

We now introduce missingness in three attributes and in five percent of the friendship network’s off-diagonal cells, and impute both jointly. The friendship network’s missing ties are updated tie-wise (the default, Section 3.5). `models` protects two hypothesised relationships: an interaction between age and department in predicting performance and, on the network side, an interaction between the sender’s age and the endogenous reciprocity term — does the tendency to reciprocate depend on the sender’s age? — which the tie-wise updater re-evaluates after every single draw:

```
attrs_miss <- attrs
set.seed(1)
attrs_miss$age[sample(n, 4)] <- NA
attrs_miss$performance[sample(n, 5)] <- NA
attrs_miss$department[sample(n, 3)] <- NA

friends_miss <- friends
off <- which(row(friends_miss) != col(friends_miss))
friends_miss[sample(off, round(length(off) * 0.05))] <- NA

## maxit = 5 (below the default of 20) keeps the tables below compact.
fit <- netmice(
  attrs_miss, list(friends = friends_miss, advice = advice),
  m = 5, maxit = 5, donors = 5, seed = 20260701, printFlag = FALSE,
  models = list("performance ~ friends_indegree + age * department",
                "friends ~ age_ego:reciprocity")
)
fit
```

```

Class: netmids
Imputations (m): 5    Iterations (maxit): 5    Method: pmm    Donors: 5
Cores: 1
Variables with missingness: age, department, performance
Networks with missingness: friends
Network-tie updating: sequential Gibbs (single-tie draws with change
statistics)
Custom models for: performance, friends

```

A completed data set is extracted with `complete_netmice()`, and convergence can be inspected via the chain means for each imputed attribute across iterations, alongside network-level diagnostics tracked at every iteration for every network:

```

completed <- complete_netmice(fit, 1)
head(completed$data)
anyNA(completed$data)
anyNA(completed$networks$friends[row(friends) != col(friends)])
round(fit$chainMean["age", , ], 2)
round(t(fit$netChain["friends", , , 1]), 3)

```

```

  age gender department performance
1  31      F      sales      56.3
2  35      M        hr      57.9
3  28      M      sales      57.6
4  13      F        hr      56.1
5  24      M        eng      51.6
6  49      M        eng      42.4
[1] FALSE
[1] FALSE
      1      2      3      4      5
1 33.75 32.25 32.00 36.00 35.25
2 31.75 36.00 32.50 28.50 33.75
3 39.00 36.25 31.00 32.25 33.25
4 34.00 29.00 29.00 32.75 33.00
5 27.50 34.00 31.25 32.25 34.75

 density reciprocity transitivity n_isolates avg_inv_geodesic
1   0.072      0.036      0.126           0      0.504
2   0.070      0.037      0.139           0      0.496
3   0.069      0.056      0.138           0      0.492
4   0.069      0.037      0.137           0      0.495
5   0.072      0.036      0.134           0      0.500

```

Both attributes and network ties are complete in the returned object, and the tracked network diagnostics are stable across iterations, consistent with (though of course not a substitute for a formal test of) convergence.

Finally, `netquickpred()` (Section 3.4) shows which predictors would enter each imputation model if only those predictors with medium correlation with the target are considered. With only $n = 40$ nodes, chance correlations of network measures easily exceed the `mincor` default of 0.1, so a strict threshold is used here purely for illustration:

```

qp <- netquickpred(attrs_miss,
  list(friends = friends_miss, advice = advice),
  targets = "performance", mincor = 0.4)
qp

```

```

Class: netquickpred (per-target predictor selection)

```

```

mincor: 0.4  steps: 3  collinearity: pairwise (threshold 0.9)
missingness screen: TRUE
Targets: performance
Dropped (missing data, not needed): department
- performance (attribute): 0 attribute(s) + 5 network feature(s)
  friends_indegree [step 1], friends_age_alter_min [step 1],
  friends_performance_alter_mean [step 1],
  friends_performance_alter_mean_in [step 1],
  friends_performance_alter_mean_out [step 1]
- age (attribute): 0 attribute(s)
- friends (network): 2 dyad term(s)
  reciprocity, twopath

```

The selection illustrates three mechanisms at once. **performance**'s own alter means — including the mean over *incoming* ties — are selected (the homophily signal discussed in Section 3.5). **age** is kept (and will be imputed, here from an intercept-only model at this strict threshold) because one of **performance**'s selected features, **friends_age_alter_min**, derives from it. And **department**, whose missing values are needed by no selected predictor of the target, is dropped from the imputation entirely. Handing **qp** (or simply **predictor_selection = "quickpred"**) to **netmice()** runs the imputation with these lean per-target models. However, if the **netmice()** call above would be repeated with this **qp** object, **department** would not be dropped, because it is part of the predictors in the **models** formula for **performance** and is thus protected.

4.1 Specifying the imputation models

When **models** is left unspecified, each imputation model is assembled automatically from the available predictors, and **netmice()** says once per session that supplying **models** is preferable. The reason is congeniality (Meng, 1994): an imputation model must be at least as rich as the analysis model that will be fitted to the completed data. Imputing without an interaction, a non-linearity, or a network effect that the substantive model will estimate draws the imputations from a distribution in which that term is absent, biasing the pooled estimate toward the null however large m is. For missing network data specifically, the consequences of an under-specified imputation model have been documented by Krause et al. (2020).

5 Discussion and future work

Signed networks are not yet supported. Several of the measures in Section 3.1 are path-based (betweenness, closeness) and require non-negative edges, while others (the weighted reciprocity and clustering formulas above) assume weights are magnitudes rather than signed valences; **netimpute** currently rejects any network with a negative tie value. Extending the package to signed networks would require, at minimum, separate treatment of positive and negative sub-graphs for the path-based measures and a structural-balance-aware notion of reciprocity and clustering, which we leave to future work. Until then, it is recommended that users interested in signed networks split their network into a positive and a negative side and use the **net_dependence** argument with **forbidden** for the positive and negative network sides.

Endogenous tie statistics. The tie-wise updater currently conditions on two endogenous statistics, reciprocity and the bounded two-path indicator, recomputed from the partly imputed matrices at every visit. Natural extensions include geometrically down-weighted shared-partner statistics in the spirit of GWESP terms from the ERGM literature (Robins et al., 2007), and degree-preserving donor matching for the simultaneous scheme. Overall, the network imputation is simple and efficient and likely sufficient in many applications. However, a proper network-model-based imputation (e.g., using ERGM or SAOM), will be superior, because it is able to incorporate many more network statistics directly into the imputation model. Obtaining such

an imputation at every iteration of the chained-equations loop is possible but enormously time consuming and is thus not implemented or suggested here.

Method tuning parameters. `netmice()`'s `method` argument dispatches generically to `mice`'s numeric-response univariate methods, globally or per target (Section 3.5), but the methods' tuning parameters (`cart`'s `minbucket`, `rf`'s `ntree`, PMM's matching type) currently keep `mice`'s defaults; exposing them — per method, or per target alongside the method itself — is a natural next step, as the internal dispatch is already method-agnostic.

6 Summary

`netimpute` provides a single, coherent R interface for a problem that is otherwise split across separate tools, jointly imputing missing attributes and missing ties through a single chained-equations loop in which ties are, by default, redrawn one at a time conditional on the current state of everything else.

References

- Burt, R. S. (1992). *Structural Holes: The Social Structure of Competition*. Harvard University Press, Cambridge, MA.
- Butts, C. T. (2008). Social network analysis with `sna`. *Journal of Statistical Software*, 24(6):1–51.
- Csardi, G. and Nepusz, T. (2006). The `igraph` software package for complex network research. In *InterJournal, Complex Systems*, page 1695.
- Gelman, A. and Rubin, D. B. (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472.
- Harrell, F. E. (2015). *Regression Modeling Strategies*. Springer, New York, 2nd edition.
- Hollway, J. (2021). *manynet: Many Ways to Make and Modify Myriad Networks*.
- Huisman, M. (2009). Imputation of missing network data: Some simple procedures. *Journal of Social Structure*, 10(1):1–29.
- Huisman, M. and Steglich, C. (2008). Treatment of non-response in longitudinal network studies. *Social networks*, 30(4):297–308.
- Koskinen, J. H., Robins, G. L., and Pattison, P. E. (2010). Analysing exponential random graph (p-star) models with missing data using bayesian data augmentation. *Statistical Methodology*, 7(3):366–384.
- Koskinen, J. H., Robins, G. L., Wang, P., and Pattison, P. E. (2013). Bayesian analysis for partially observed network data, missing ties, attributes and actors. *Social networks*, 35(4):514–527.
- Krackhardt, D. (1988). Predicting with networks: Nonparametric multiple regression analysis of dyadic data. *Social Networks*, 10(4):359–381.
- Krause, R. W. (2019). *Multiple Imputation for Missing Network Data*. Doctoral dissertation, University of Groningen, Groningen, The Netherlands.
- Krause, R. W., Huisman, M., and Snijders, T. A. B. (2018). Multiple imputation for longitudinal network data. *Statistica Applicata-Italian Journal of Applied Statistics*, 30(1):33–57.

- Krause, R. W., Huisman, M., Steglich, C., and Snijders, T. A. B. (2020). Missing data in cross-sectional networks – An extensive comparison of missing data treatment methods. *Social Networks*, 62:99–112.
- Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*. Wiley, Hoboken, NJ, 2nd edition.
- Meng, X.-L. (1994). Multiple-imputation inferences with uncongenial sources of input. *Statistical Science*, 9(4):538–558.
- Opsahl, T. and Panzarasa, P. (2009). Clustering in weighted networks. *Social Networks*, 31(2):155–163.
- Peduzzi, P., Concato, J., Kemper, E., Holford, T. R., and Feinstein, A. R. (1996). A simulation study of the number of events per variable in logistic regression analysis. *Journal of Clinical Epidemiology*, 49(12):1373–1379.
- Robins, G., Pattison, P., Kalish, Y., and Lusher, D. (2007). An introduction to exponential random graph (p^*) models for social networks. *Social Networks*, 29(2):173–191.
- Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley, New York.
- Snijders, T. A. (1996). Stochastic actor-oriented models for network change. *Journal of mathematical sociology*, 21(1-2):149–172.
- Snijders, T. A. (2017). Stochastic actor-oriented models for network dynamics. *Annual review of statistics and its application*, 4(1):343–363.
- van Buuren, S. and Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3):1–67.
- Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., and Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 16(2):667–718.
- Venables, W. N. and Ripley, B. D. (2002). *Modern Applied Statistics with S*. Springer, New York, 4th edition.